SC2001 ALGORITHM DESIGN AND ANALYSIS · PROJECT 3

Dynamic programming for the unbounded knapsack

Recursive definition · subproblem graph · bottom-up algorithm · results

[Group name] · [Member names] · [Lab group]

THE PROBLEM

Packing the most profit into capacity C

A knapsack holds total weight at most **C**. There are **n** object types; type *i* has weight w_i and profit p_i, all positive integers.

Every type has an **unlimited supply**, so the same type can be packed any number of times.

P(C) = the largest total profit of any set of objects that fits in capacity C.

Instance A · used in Q2 and Q4a

i	0	1	2
w_i	4	5	8
p_i	7	6	9

Instance B · used in Q4b

i	0	1	2
w_i	5	6	7
p_i	7	6	9

QUESTION 1

Recursive definition of P(C)

$$P(0) = 0$$

$$P(C) = \max(0, \max\{i : w_i \leq C\} [P(C - w_i) + p_i])$$

$P(C - w_i) + p_i$

Put one type-i object in, then pack the leftover space $C - w_i$ as well as possible.

max over i with $w_i \leq C$

We don't know which type the best packing uses, so we try every type that fits.

max(0, ...)

If no type fits (C is smaller than every w_i), the knapsack stays empty and the profit is 0.

QUESTION 1

Why the recurrence is correct

Optimal substructure

Take an optimal packing for C and remove one type-i object. What is left must be optimal for C – w_i, or a better packing there would beat the optimum.

Unlimited supply

Type i is still available after one is used, so the subproblem has the same form. Only the capacity shrinks; no record of used items is needed.

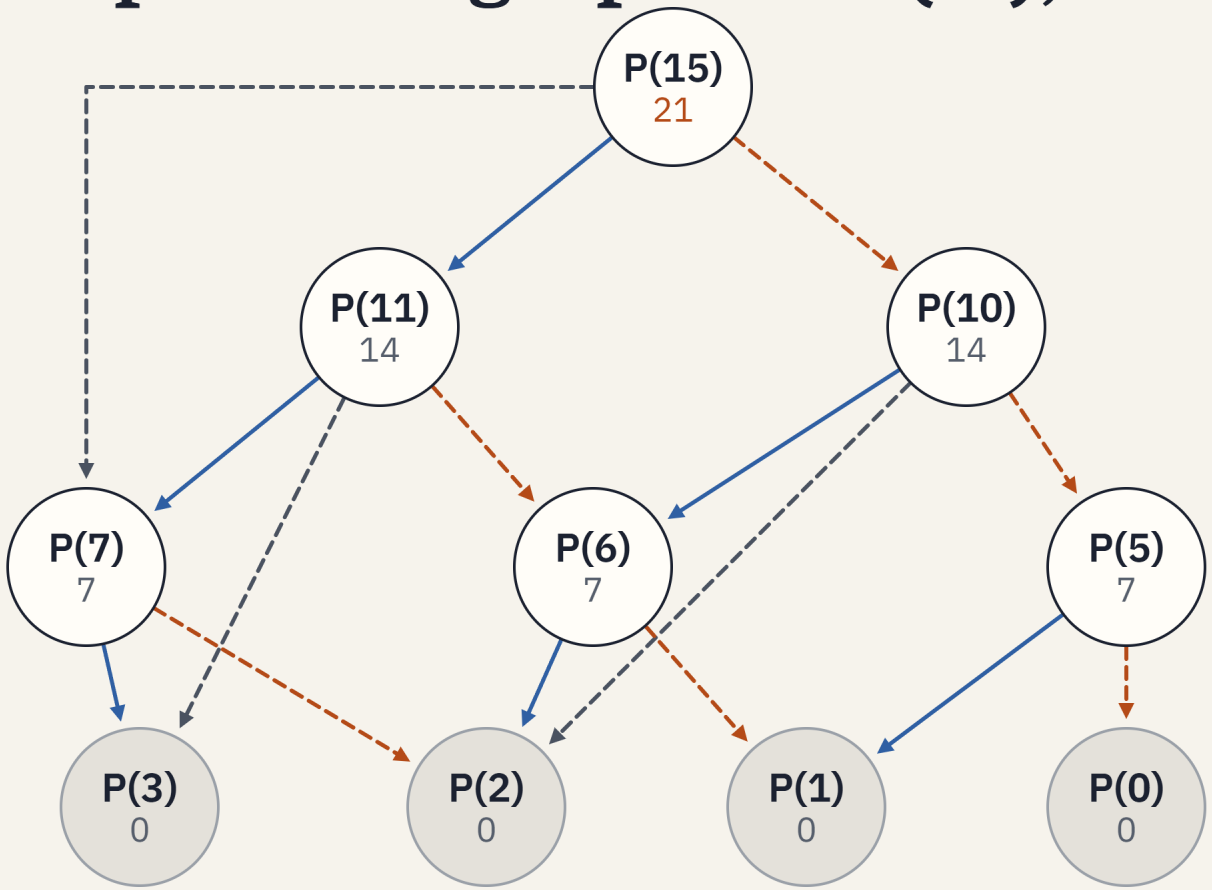
It always ends

Each step lowers the capacity by some $w_i \geq 1$, so every chain of calls reaches a capacity where nothing fits.

Example: P(15) = max(P(11) + 7, P(10) + 6, P(7) + 9) = max(21, 20, 16) = **21**

QUESTION 2

Subproblem graph for P(15), instance A



Reading the graph

Each node is a subproblem with its value below. An edge P(c) → P(c − w_i) means P(c) depends on it.

- −4 · take type 0 (p 7)
- −5 · take type 1 (p 6)
- −8 · take type 2 (p 9)

Gray nodes are base cases: nothing fits, so P = 0.

QUESTION 2

What the subproblem graph tells us

10

distinct subproblems

Capacities 15, 11, 10, 7, 6, 5, 3, 2, 1 and 0. A plain recursion tree would repeat many of them.

5

shared subproblems

P(7), P(6), P(3), P(2) and P(1) each have more than one parent: the subproblems overlap.

0

cycles

Every edge lowers the capacity, so solving in increasing c always has the inputs ready.

Answer: **P(15) = 21**, from three objects of type 0 (weight 12, profit 21). The next best, 4 + 4 + 5, gives 20.

QUESTION 3

Bottom-up dynamic programming algorithm

```
MaxProfit(C, w[0..n-1], p[0..n-1]):
    P[0..C] ← 0
    for c ← 1 to C:
        for i ← 0 to n-1:
            if w[i] ≤ c and P[c - w[i]] + p[i] > P[c]:
                P[c] ← P[c - w[i]] + p[i]
    return P[C]
```

Fill the table from **small c to large c**.

When P[c] is computed, every P[c – w_i] is already final, because c – w_i < c.

Each entry evaluates the Question 1 recurrence **exactly once**. Starting at 0 covers “nothing fits”.

QUESTION 3

Cost of the bottom-up algorithm

O(nC)

time

C table entries, and each one checks all n object types once.

O(C)

space

One 1-D table P[0..C]; the trace-back array is the same size.

Why one dimension is enough

Unlimited supply means P[c] never needs to know which items were already used. The 0/1 knapsack, where each item is used once, needs a 2-D table P[i][c].

Pseudo-polynomial

O(nC) is polynomial in the value of C, but C needs only log C bits to write down, so the running time is exponential in the input size.

QUESTION 4

Implementation in Python

```
def max_profit(C, w, p):
    n = len(w)
    P = [0] * (C + 1)        # P[c] = best profit
    choice = [-1] * (C + 1)  # last type added
    for c in range(1, C + 1):
        for i in range(n):
            if w[i] ⩽ c and P[c - w[i]] + p[i] > P[c]:
                P[c] = P[c - w[i]] + p[i]
                choice[c] = i
    return P, choice
```

A direct Python translation of the Question 3 pseudocode.

choice[c] remembers the last type added, so following it from C back to 0 lists the objects used.

Run with python3 knapsack.py

QUESTION 4A

P(16) for w = (4, 5, 8), p = (7, 6, 9)

28

P(16) = 28: four objects of type 0

Type 0 has the best profit per unit weight (7/4 = 1.75, against 1.20 and 1.125), and 16 = 4 × 4, so four of them fill the knapsack exactly.

c	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
P(c)	0	0	0	0	7	7	7	7	14	14	14	14	21	21	21	21	28

QUESTION 4B

P(12) for w = (5, 6, 7), p = (7, 6, 9)

16

P(12) = 16: one type 0 and one type 2

Weights 5 + 7 = 12 fill the knapsack exactly, for profit 7 + 9 = 16.

c	0	1	2	3	4	5	6	7	8	9	10	11	12
P(c)	0	0	0	0	0	7	7	9	9	9	14	14	16

Greedy by ratio

Best ratio first (type 0, 1.4): 5 + 5 = 10, profit 14, and 2 units of space wasted.

Dynamic programming

Tries every combination through its subproblems: 5 + 7 = 12, profit 16.

Summary

Q1	$P(C) = \max$ over fitting types of $P(C - w_i) + p_i$, with $P = 0$ when nothing fits.
Q2	10 overlapping subproblems in an acyclic graph; $P(15) = 21$.
Q3	Fill $P[0..C]$ bottom-up: $O(nC)$ time, $O(C)$ space.
Q4	$P(16) = 28$ with instance A; $P(12) = 16$ with instance B.

Thank you. Questions?